UNIVERSITÉ NATIONALE DES SCIENCES, TECHNOLOGIES,
INGÉNIERIE ET MATHÉMATIQUES (UNSTIM)

FACULTÉ DES SCIENCES ET TECHNIQUES DE NATITINGOU (FAST)

RAPPORT DE TRAVAUX PRATIQUES

CONCEPTION ET RÉALISATION D'UNE APPLICATION DE GESTION DES RÉSERVATIONS DE VOLS

Filière : Mathématiques et Informatique
Niveau : Licence 3
Groupe : 12
Outils utilisés : C++, PlantUML et GitHub
Étudiant : .....

Année académique : 2026–2027

SOMMAIRE

1. Introduction

2. Présentation du problème et objectifs

3. Analyse des besoins et règles de gestion

4. Modélisation UML

5. Implémentation en C++

6. Fonctionnement de l’application

7. Compilation, vérification et limites

8. Conclusion

1. Introduction

Dans le cadre des travaux pratiques, il nous a été demandé de concevoir et de réaliser une application permettant de gérer les réservations de billets d'avion pour une agence de voyage située à Cotonou. Ce travail nous a permis de mettre en pratique les notions de programmation orientée objet, de modélisation UML et de programmation en C++.

L'objectif de ce rapport est de présenter les différentes étapes que j'ai suivies pour analyser le problème, identifier les classes nécessaires, représenter leurs relations et réaliser une première application en mode console.

2. Présentation du problème et objectifs

La gestion manuelle des réservations peut rendre difficile le suivi des passagers, des vols, des paiements et des annulations. L'application proposée vise à organiser ces informations et à faciliter les opérations courantes de l'agence.

2.1 Objectif général

Concevoir une application simple qui permet de gérer les passagers, les vols, les réservations, les paiements et les remboursements.

2.2 Objectifs spécifiques

• Enregistrer les informations des passagers.

• Ajouter et consulter les vols disponibles.

• Rechercher un vol selon les informations saisies.

• Effectuer et annuler une réservation.

• Enregistrer un paiement et vérifier son état.

• Consulter les informations relatives à une réservation.

• Prendre en compte le traitement d'un remboursement lorsqu'une réservation payée est annulée.

3. Analyse des besoins et règles de gestion

Avant de commencer la programmation, j'ai relevé les principales opérations que le système doit permettre à l'utilisateur d'effectuer. L'application est organisée autour d'un menu principal en mode console.

3.1 Fonctionnalités prévues

N°	Fonctionnalité
1	Enregistrer un passager
2	Ajouter un vol
3	Afficher les vols disponibles
4	Rechercher un vol
5	Effectuer une réservation
6	Enregistrer un paiement
7	Annuler une réservation

N°	Fonctionnalité
8	Consulter les réservations
9	Afficher les informations d'une réservation
10	Quitter l'application

3.2 Règles de gestion

• Un passager doit être enregistré avant de pouvoir effectuer une réservation.

• Un vol ne doit pas dépasser sa capacité maximale de places.

• Un même passager ne peut pas réserver deux fois le même vol.

• Chaque réservation doit être liée à un seul passager et à un seul vol.

• Une réservation annulée ne peut pas faire l'objet d'un paiement.

• La place est considérée comme définitivement réservée après validation du paiement.

• Une réservation payée puis annulée doit être traitée selon les règles de remboursement fixées par l'agence.

La consigne ne précise pas de taux ni de montant de remboursement. Il n'est donc pas possible de fixer une formule de remboursement sans disposer des règles propres à l'agence.

4. Modélisation UML

La modélisation UML m'a permis de représenter la structure générale de l'application avant de passer à la programmation. Après analyse du problème, j'ai retenu cinq classes principales : Passager, Vol, Reservation, Paiement et Remboursement.

4.1 Description des classes

Classe	Rôle dans l'application
Passager	Contient l'identification et les coordonnées d'un client.
Vol	Décrit un vol : numéro, destination, date, heure, capacité et prix.
Reservation	Relie un passager à un vol et conserve les informations de réservation.
Paiement	Représente le règlement associé à une réservation.
Remboursement	Représente le remboursement éventuel après annulation d'une réservation payée.

4.2 Associations et multiplicités

• Un Passager peut effectuer zéro, une ou plusieurs Reservations ; chaque Reservation concerne un seul Passager.

• Un Vol peut être concerné par zéro, une ou plusieurs Reservations ; chaque Reservation concerne un seul Vol.

• Une Reservation reçoit zéro ou un Paiement ; un Paiement est rattaché à une réservation.

• Un Paiement peut donner lieu à zéro ou un Remboursement.

Le diagramme de classes a été réalisé avec PlantUML. Le fichier source diagramme_classes.puml permet de modifier le diagramme, tandis que le fichier diagramme_classes.png permet de le consulter directement.

Emplacement du diagramme : le diagramme de classes est joint au dépôt GitHub du projet sous le nom diagramme_classes.png.

5. Implémentation en C++

Après la modélisation, j'ai réalisé une première implémentation en C++. Le programme applique les principes de la programmation orientée objet : les données sont regroupées dans des classes et l'accès aux informations est organisé au moyen des méthodes prévues.

5.1 Organisation du programme

Le code source principal est enregistré dans le fichier main.cpp. Il contient les classes nécessaires à la gestion des passagers, des vols, des réservations, des paiements et des remboursements, ainsi que la fonction main() qui affiche le menu et dirige les opérations selon le choix de l'utilisateur.

5.2 Attributs et méthodes

Chaque classe possède des attributs qui décrivent ses données et des méthodes qui permettent d'effectuer les opérations correspondantes. Par exemple, la classe Passager contient les informations d'identification du passager et des méthodes d'enregistrement et d'affichage. La classe Vol contient les informations du vol et permet notamment de vérifier la disponibilité.

5.3 Stockage des données

Dans cette version, des collections de type vector sont utilisées pour conserver les objets créés pendant l'exécution du programme. Cette solution permet de gérer plusieurs passagers, vols et réservations sans créer une variable séparée pour chaque élément. Les données restent toutefois en mémoire pendant l'exécution : elles ne sont pas conservées automatiquement après la fermeture du programme.

6. Fonctionnement de l'application

Au lancement, le programme affiche un menu principal. L'utilisateur choisit une opération en saisissant le numéro correspondant. Le programme exécute ensuite l'action demandée et revient au menu afin de permettre une nouvelle opération.

Le déroulement normal consiste notamment à enregistrer d'abord un passager, ajouter un vol, puis effectuer une réservation. Le paiement est enregistré séparément. Cette organisation permet de tenir compte de la règle selon laquelle une place n'est définitivement réservée qu'après validation du paiement.

L'annulation d'une réservation est également prévue. Lorsqu'une réservation payée est annulée, la question du remboursement doit être traitée conformément à la politique de l'agence. Le montant et les conditions exactes doivent être définis par cette dernière.

7. Compilation, vérification et limites

Le programme a été compilé et lancé dans l'environnement de développement utilisé pour le TP. J'ai effectué une première vérification de son fonctionnement et constaté que le programme s'exécute. Le code source a ensuite été ajouté au dépôt GitHub du groupe.

Cette version constitue une application console pédagogique. Elle ne dispose pas encore d'une base de données ni d'un mécanisme de sauvegarde permanente. Une amélioration possible serait d'ajouter l'enregistrement des données dans des fichiers ou dans une base de données afin de les retrouver après la fermeture de l'application.

Une autre amélioration serait de préciser avec l’agence les règles de remboursement, puis de les intégrer de manière complète au programme. Ces précisions permettraient également de renforcer la gestion des paiements et des annulations.

8. Conclusion

Ce travail pratique m’a permis de mettre en application les notions étudiées en programmation orientée objet et en modélisation UML. J’ai commencé par analyser le problème et identifier les classes principales, puis j’ai représenté leurs relations dans un diagramme de classes. J’ai ensuite réalisé une application en C++ organisée autour d’un menu qui regroupe les principales opérations de gestion des vols et des réservations.

Ce TP m’a également permis de mieux comprendre l’intérêt de concevoir un système avant de commencer sa programmation, ainsi que le rôle des classes, des objets, des associations et des collections. Le projet peut être amélioré par l’ajout d’une sauvegarde permanente des données et par une définition plus précise des règles de remboursement.